Practical - 13

Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Theory

1. Do PHP endpoints return valid JSON responses?

Answer: Yes, PHP API endpoints should return valid JSON responses when the frontend communicates with the backend. JSON (JavaScript Object Notation) is a lightweight data format used to exchange data between applications. PHP can convert arrays or objects into JSON using the json_encode() function. The API should also set the appropriate Content-Type: application/json header and return consistent responses for successful operations and errors.

Reference Link:
https://www.w3schools.com/php/php_json.asp

2. Are GET and POST requests handled properly?

Answer: Yes, GET and POST requests should be handled according to their purposes. The GET method is used to retrieve event or student records from the database, while POST is used to submit new data to the server. For update and delete operations, the API should use appropriate request handling, such as PUT/PATCH and DELETE, or POST with an action parameter in a simpler PHP implementation. The PHP backend checks the request method, processes the input, performs the required database operation, and returns a JSON response

Reference Link:
https://www.w3schools.com/php/php_forms.asp
https://www.w3schools.com/TAGs/ref_httpmethods.asp

3. Are loading, success, and error states shown on the frontend?

Answer: Yes, the frontend should display suitable loading, success, and error states while communicating with the PHP API. When a Fetch request starts, a loading message can be displayed. After the request completes successfully, the interface can show a

confirmation message and update the displayed records without reloading the entire page. If the request fails or the server returns an error, an appropriate error message should be displayed. JavaScript's Fetch API and Promise handling help manage these asynchronous operations.

Reference Link:
https://www.w3schools.com/js/js_api_fetch.asp
https://www.w3schools.com/JS/js_ajax_intro.asp

4. Is input validation performed before database operations?

Answer: Yes, input validation should be performed on the server side before any database operation. The PHP API should check that required fields are present, values are in the correct format, and inputs meet the required length and range restrictions. Invalid data should be rejected with an appropriate error response. Prepared statements should be used for database queries to help prevent SQL injection, and output should be safely handled to reduce cross-site scripting (XSS) risks. Frontend validation can improve usability, but it must not replace server-side validation

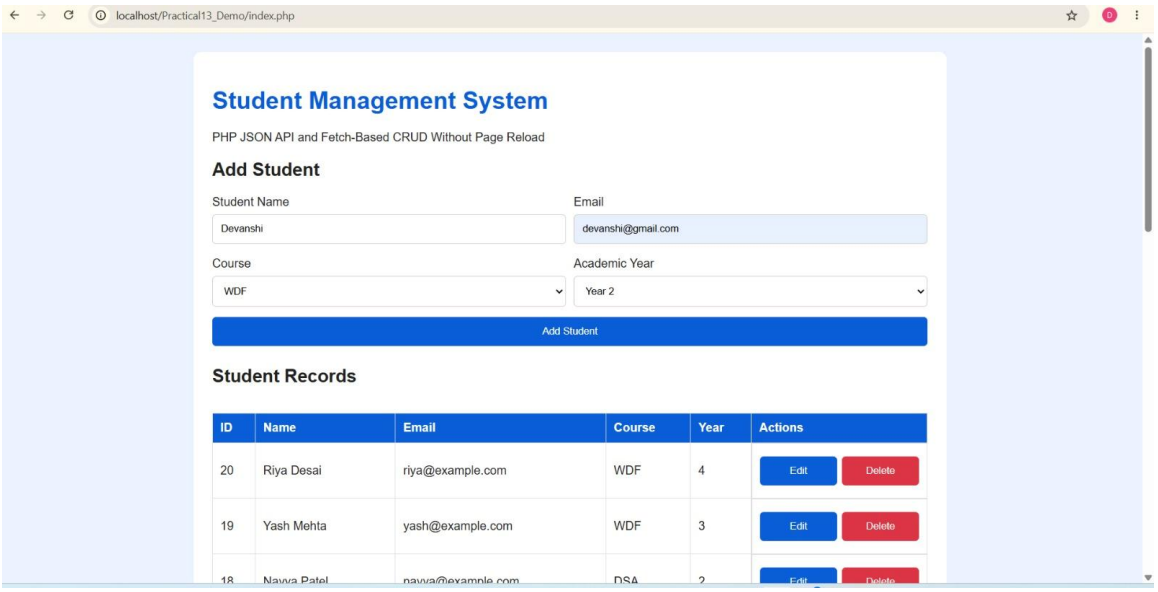
Reference Link:
https://www.w3schools.com/php/php_form_validation.asp
https://www.w3schools.com/php/php_mysql_prepared_statements.asp

A. Implementation/ Output/Result

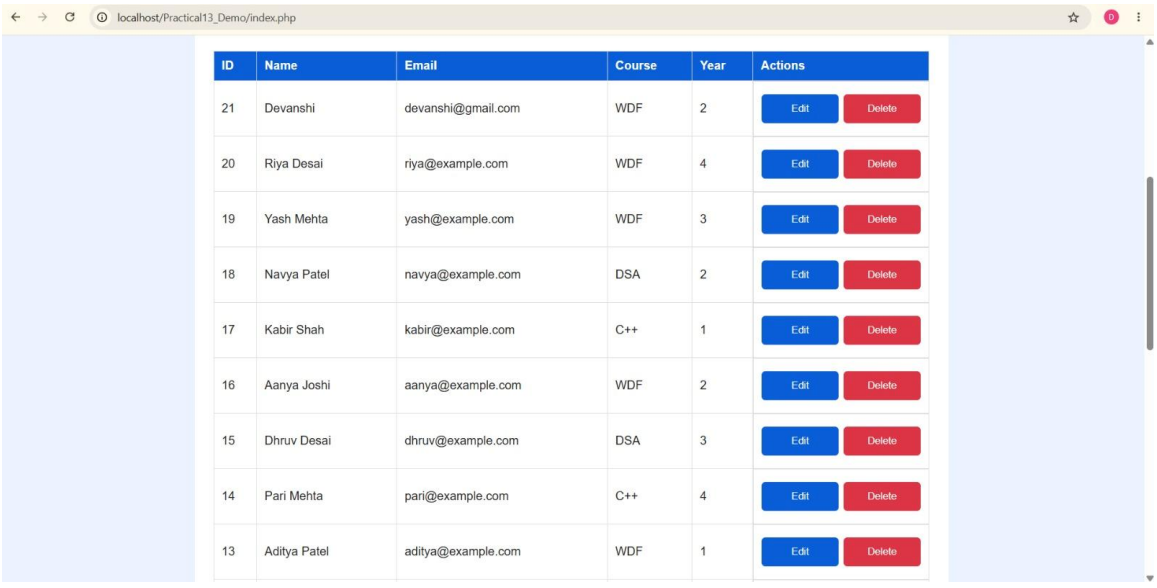
```
B.
C. <?php
D. mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
E.
F. try {
G.     $conn = new mysqli(
H.         "localhost",
I.         "root",
J.         "",
K.         "student_management"
L.     );
M.
N.     $conn->set_charset("utf8mb4");
O. } catch (mysqli_sql_exception $e) {
P.     error_log($e->getMessage());
Q.     http_response_code(500);
R.     exit("Database connection failed.");
S. }
```

```
T.  ?>
U.
```

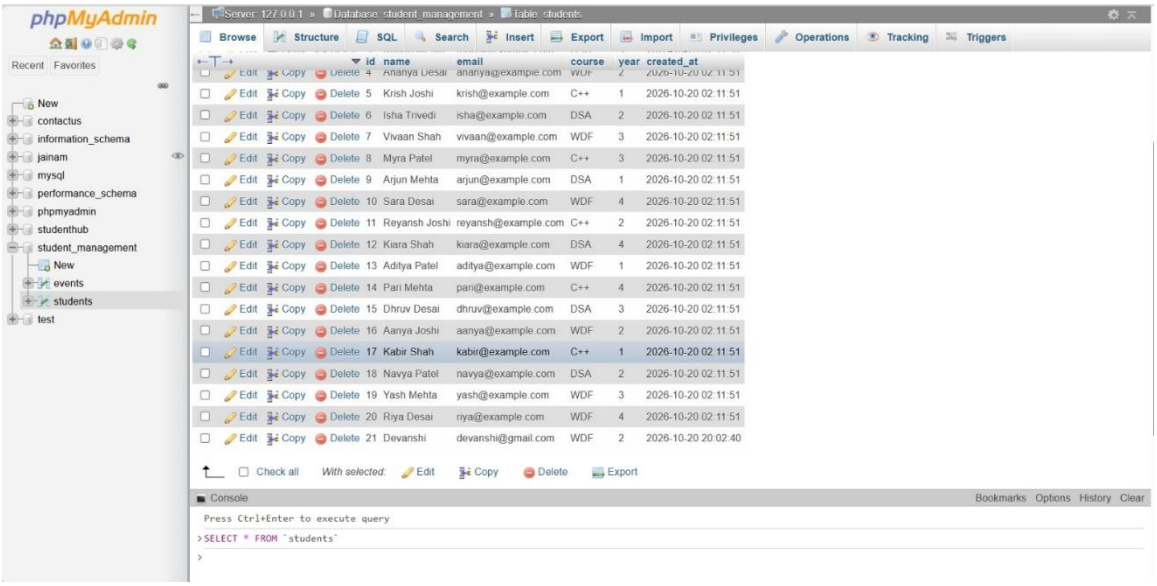
B. Evidence of Work (Screenshots)



Description: This screenshot demonstrates the frontend of the Student Management System implemented using PHP JSON API and Fetch API. It provides an Add Student form and displays student records dynamically in a table. The Edit and Delete buttons support asynchronous update and delete operations. The interface demonstrates frontend-backend integration and CRUD functionality without requiring a full page reload.



Description: This screenshot demonstrates the dynamic display of student records using the Fetch API. The records are retrieved from the PHP JSON API and displayed without requiring a complete page reload. The Edit and Delete controls allow users to perform asynchronous CRUD operations, demonstrating the interactive nature of the API-driven application.



Description: This screenshot demonstrates the backend database of the Event Management System using phpMyAdmin. The events table stores event information such as ID, title, description, date, venue, status, poster, and creation timestamp. The records shown in the database correspond to the events displayed in the admin interface, confirming successful integration between the PHP application and MySQL database.

GitHub Repository Link:

https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical13_Demo

C. Challenges Faced:

- JSON response handling: Ensuring that PHP returns valid JSON without unwanted HTML, warnings, or error messages.
- Frontend-backend integration: Connecting JavaScript Fetch requests with PHP API endpoints and processing the returned data correctly.
- Asynchronous operations: Managing loading states and ensuring that the interface updates only after the server response is received.

- HTTP request handling: Handling GET and POST requests correctly and implementing update and delete operations.

- Input validation: Checking required fields and rejecting invalid data before performing database operations.

- Database errors: Handling connection failures, SQL errors, and unsuccessful insert, update, or delete operations.

- Error handling and debugging: Identifying incorrect API paths, HTTP errors, malformed JSON, and network issues using the browser's Developer Tools and Network tab

D. Learning Outcomes:

- Create PHP API endpoints that return JSON responses.

- Connect PHP with a MySQL database to manage student or event records.

- Use JavaScript Fetch API to send asynchronous HTTP requests.

- Implement CRUD operations without reloading the entire webpage.

- Handle GET and POST requests and understand the purpose of other HTTP methods.

- Parse JSON responses and dynamically update frontend content using JavaScript.

- Display loading indicators, success messages, and error messages.

- Perform server-side input validation and use prepared statements for safer database operations.

- Debug API requests and inspect responses using the browser's Network tab.

- Understand how frontend JavaScript and backend PHP work together in modern web applications.

Faculty Remarks: (Keep this blank for faculty)